

Stats

▼ More

★ Member-only story

Everything from structure and triggers to testing and distribution.

8 min read · Apr 17, 2026

...

The Complete Guide to Building Skills for Claude

- ☐ Reading list
- ☐ Implement
- ☐ Media Generation
- ☐ Python
- ☐ Business Opportunities

Create new list

Non-members, read [here](#).

Page 1 of 10

Then Anthropic released a complete internal guide. 30 pages. Every pattern, every gotcha, every checklist. I read the whole thing.

Here's what actually matters.

What a skill even is

A skill is a folder. That's it.

Inside that folder is a `SKILL.md` file — the main instruction file — and optionally some scripts, reference docs, or asset templates. You zip the folder, upload it to Claude.ai via Settings > Capabilities > Skills, and from that point on Claude knows how to handle whatever workflow you've described.

The idea is simple: instead of re-explaining your process every single conversation, you teach Claude once. Sprint planning, document generation, onboarding workflows, API integrations — anything repeatable becomes a skill.

The folder structure looks like this:

```
your-skill-name/  
├── SKILL.md           ← required  
├── scripts/          ← optional  
├── references/        ← optional  
└── assets/           ← optional
```

Skills work identically across Claude.ai, Claude Code, and the API. Build it once, use it everywhere.

The three-level loading system

This is the part most people miss, and it explains a lot about why some skills work and others don't.

Skills load in three levels:

Level 1 — YAML frontmatter. Always present in Claude's system prompt, for every conversation. This is where Claude decides whether your skill is relevant. It's just a few lines, which is the point — it keeps token usage low.

Level 2 — The `SKILL.md` body. Loaded when Claude thinks the skill applies.

This is where your actual instructions live.

Level 3 — Linked files in references/. Only loaded if Claude needs them.
Deep documentation, API patterns, edge case guides.

The consequence of this system: your frontmatter description is the most important thing in your entire skill. If it's bad, Claude never loads the skill. The instructions don't matter if nobody reads them.

Writing the SKILL.md file

The file has two parts: a YAML header (frontmatter) and the body (your actual instructions).

The frontmatter

Minimal valid format:

```
---
name: your-skill-name
description: What it does and when to use it.
---
```

The `name` field must be kebab-case. No spaces, no capitals, no underscores.
`my-cool-skill` works. `My Cool Skill` does not.

The `description` field is where most people go wrong. It must include two things: what the skill does, and when to use it. Both. Not one.

Examples of good descriptions:

```
Analyzes Figma design files and generates developer handoff documentation.
Use when user uploads .fig files, asks for "design specs",
"component documentation", or "design-to-code handoff".
```

```
Manages Linear project workflows including sprint planning, task creation,
and status tracking. Use when user mentions "sprint", "Linear tasks",
"project planning", or asks to "create tickets".
```

Examples of descriptions that won't work:

Helps **with** projects.

Creates sophisticated multi-page documentation systems.

The first is too vague. The second has no trigger phrases — Claude has no idea when to load it.

You can also add optional fields:

```
---
name: my-skill
description: ...
license: MIT
metadata:
  author: Your Name
  version: 1.0.0
  mcp-server: your-server-name
---
```

One hard rule: no XML angle brackets anywhere in the frontmatter. They're a security restriction because frontmatter goes directly into Claude's system prompt.

The instructions body

After the frontmatter, write instructions in plain Markdown. The guide recommends a structure like this:

```
# Skill Name

## Step 1: First thing
Clear explanation. What to do, what success looks like.

## Step 2: Next thing
...
## Examples
User says: "Set up a new project"
Actions:
1. Fetch existing projects via MCP
2. Create new project with provided parameters
Result: Project created with confirmation link
## Troubleshooting
Error: Connection refused
Cause: MCP server isn't running
Solution: Settings > Extensions > [Service] > Reconnect
```

Be specific. "Validate the data before proceeding" tells Claude nothing. "Run `python scripts/validate.py --input {filename}` and if it fails, check for missing required fields or date formats" is something Claude can actually act on.

Keep SKILL.md focused and move deep documentation to `references/`. Link to it from the instructions. That's how the progressive disclosure system is supposed to work.

The three skill categories

Anthropic identified three main use cases from early builders:

Category 1: Document and asset creation. Skills that generate consistent output — documents, presentations, code, designs. The frontend-design skill falls here. Key pattern: embed style guides, use templates, run quality checks before finishing. No external tools needed.

Category 2: Workflow automation. Multi-step processes with consistent methodology. The skill-creator skill itself is the example here. Key pattern: step-by-step with validation gates, iterative refinement loops, built-in improvement suggestions.

Category 3: MCP enhancement. Workflow guidance layered on top of an existing MCP server connection. Sentry's code-review skill is the example — it uses Sentry's MCP to analyze bugs in GitHub PRs. Key pattern: sequences multiple MCP calls, embeds domain expertise, handles common errors.

Skills + MCP

If you already have an MCP server connected to Claude, skills are the missing piece.

The guide uses a kitchen analogy. MCP provides the kitchen — the tools, the ingredients, access to everything. Skills provide the recipes — the step-by-step instructions for what to actually make.

Without skills, users connect your MCP server and then don't know what to do. They prompt inconsistently, get inconsistent results, and eventually blame the connector.

With skills, the workflow activates automatically. Claude knows the sequence. The domain expertise is embedded.

Anthropic identified five patterns that work well for MCP-enhanced skills:

Pattern 1: Sequential workflow orchestration. When steps must happen in order. Create account → set up payment → create subscription → send welcome email. Each step validates before moving to the next, with rollback instructions if something fails.

Pattern 2: Multi-MCP coordination. When the workflow spans multiple services. Export from Figma → upload to Drive → create tasks in Linear → notify on Slack. Clear phase separation, data passing between services, centralized error handling.

Pattern 3: Iterative refinement. When quality improves with iteration. Generate first draft → validate → identify issues → fix → re-validate → repeat until threshold met. Works well for reports, documents, anything with defined quality criteria.

Pattern 4: Context-aware tool selection. When the same outcome needs different tools depending on what's being handled. File over 10MB? Use cloud storage. Collaborative doc? Use Notion. Code file? Use GitHub. The skill makes the decision and explains why to the user.

Pattern 5: Domain-specific intelligence. When the skill adds specialized knowledge beyond just running tools. A payment processing skill that runs compliance checks before transactions. A legal skill that applies jurisdiction rules. The expertise is embedded in the logic, not left to the user.

Testing your skill

The guide breaks testing into three areas.

Triggering tests. Does your skill load when it should? Run 10–20 queries that should trigger it and track how many actually do. Also test that it doesn't trigger on unrelated queries. Aim for 90% accuracy on relevant queries, 0% on clearly unrelated ones.

Functional tests. Does the skill produce correct output? Run the same request 3–5 times and compare results. Count tool calls. Monitor for API errors.

Performance comparison. Is the skill actually better than not having it? Compare token usage, number of back-and-forth messages, and failed API

calls with and without the skill active.

There's also a practical debugging trick in the guide: ask Claude directly, "When would you use the [skill name] skill?" Claude will quote the description back at you. If the answer reveals gaps, you know exactly what to fix.

One pattern that works well for iteration: get Claude to succeed at a single hard task first, then extract that approach into a skill. Don't build broad coverage from day one. Get one thing working reliably, then expand.

Common failures and fixes

Skill won't upload. Usually a naming issue. The file must be exactly `SKILL.md` — case-sensitive, no variations. The folder must be kebab-case. If you see "Invalid frontmatter," check your YAML delimiters (the `---` lines) and make sure no quotes are unclosed.

Skill doesn't trigger. Description is too vague or missing trigger phrases. Add specific language users would actually say. Ask Claude what triggers the skill — its answer will show you what's missing.

Skill triggers too much. Add negative triggers explicitly: "Do NOT use for X — use the Y skill instead." Be more specific about scope.

MCP calls fail but skill loads. Test the MCP connection independently first. Ask Claude to call the MCP tool directly without the skill. If that fails, the issue is authentication or the MCP server, not your skill.

Claude loads the skill but ignores the instructions. A few causes: instructions are too long (move details to `references/`), critical steps are buried (put them at the top), language is too ambiguous (be explicit about what "validate" means). For truly critical checks, consider a validation script instead — code is deterministic, natural language isn't.

Context feels slow or degraded. Keep `SKILL.md` under 5,000 words. If you have more than 20–50 skills enabled simultaneously, consider trimming. Move detailed documentation to `references/` and link to it.

Distributing your skill

Right now (as of early 2026), the distribution flow is:

1. Host the skill on GitHub — public repo, clear README for human visitors, example usage with screenshots
2. Add the skill to your MCP documentation if you have one
3. Users download the folder, zip it, upload via Settings > Capabilities > Skills

Organization admins can also deploy skills workspace-wide, with automatic updates and centralized management — that shipped in December 2025.

For teams building on the API, skills are accessible via the `/v1/skills` endpoint and can be attached to Messages API requests via the `container.skills` parameter. They require the Code Execution Tool beta.

When writing about your skill — in the README or anywhere else — describe outcomes, not mechanics. “Enables teams to set up complete project workspaces in seconds instead of spending 30 minutes on manual setup” lands better than “a folder containing YAML frontmatter and Markdown instructions.”

The checklist (condensed)

Before you start: identify 2–3 concrete use cases. Know which tools are needed.

During development: kebab-case folder name. Exact `SKILL.md` spelling. YAML with `---` delimiters. Description includes both what and when. No XML tags anywhere. Clear, specific instructions with error handling and examples.

Before uploading: test triggering on obvious and paraphrased queries. Verify it doesn't trigger on unrelated topics. Test the actual workflow.

After uploading: monitor for under/over-triggering. Collect feedback. Iterate on description and instructions.

The thing that stands out reading this whole guide is how much weight sits on a single field — the description. Everything else can be refined. But if the description doesn't tell Claude when to load the skill, none of it matters.

Get that right first. Everything else follows from there.

If you are looking for reading the complete guide, here it is — [The Complete](#)

Guide to building skills for Claude

This story is published on [Generative AI](#). Connect with us on [LinkedIn](#) and follow [Zeniteq](#) to stay in the loop with the latest AI stories.

Subscribe to our [newsletter](#) and [YouTube](#) channel to stay updated with the latest news and updates on generative AI. Let's shape the future of AI together!

- Claude
- Claude Skills
- Productivity
- Generative Ai Tools
- Artificial Intelligence



Published in Generative AI

86K followers · Last published 2 days ago

[Following](#)

Stay updated with the latest news, research, and developments in the world of generative AI. We cover everything from AI model updates, comprehensive tutorials, and real-world applications to the broader impact of AI on society. Work with us: jimclydegm@gmail.com



Written by Ruqaiya Beguwala

330 followers · 45 following

[Follow](#)

Software Engineer | Writer | Hustler

